SWEOps-Bench: Evaluating Agents for End-to-End Resolution of Software-Induced Cloud Incidents

ANONYMOUS AUTHOR(S)

As autonomous agents advance, the traditional boundary between software development and cloud operations gradually blurs, increasing the need for integrated incident detection, localization, and remediation. Existing benchmarks evaluate isolated diagnostic queries or repository-level program repair, failing to test continuous incident resolution across operational clusters and source repositories. We introduce SWEOps-Bench, an interactive benchmark comprising 55 reproducible incident tasks derived from confirmed historical defects across 16 Kubernetes operator projects. Each task preserves the original software repository and initializes in a healthy cluster before a controlled workload triggers the historical fault. Agents investigate runtime evidence and submit source or configuration repairs through a terminal interface, while the harness builds and deploys the changes within a shared time budget.

Evaluation covers stage-wise accuracy, verified recovery, penalized lifecycle latency, and token consumption. Across six agent–model configurations, faulty-state detection ranges from 72.7% to 90.9%, whereas overall recovery ranges from only 14.5% to 52.7%, exposing a substantial gap between recognizing incidents and resolving them end to end. Trajectory analysis further shows that reported diagnosis and final recovery only partially coincide: some tasks achieve verified recovery despite an incorrect reported RCA, while runtime symptom clearance can still fail fix-specific functional verification. SWEOps-Bench establishes a rigorous foundation for evaluating autonomous agents in integrated development and operational environments.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; *Cloud computing*; • **Computing methodologies → Intelligent agents**.

Additional Key Words and Phrases: Cloud incident resolution, LLM-based agents, root cause analysis, automated remediation, agent benchmarking

1 Introduction

Modern cloud systems underpin critical digital services, and incidents can disrupt service availability and impose substantial operational costs [5, 9, 42]. When incidents originate from software defects, resolving them requires both operational investigation and changes to the underlying implementation. LLM-based agents that inspect running systems and modify source code create an opportunity to integrate development and operations within a continuous incident-resolution workflow. Kubernetes operators provide a representative setting for this integration, where software defects directly impair managed cloud workloads [10, 11]. This resolution lifecycle couples runtime anomaly detection, root cause analysis (RCA), and remediation. A central question is whether autonomous agents can resolve software-induced cloud incidents end to end within practical time.

To illustrate this integrated resolution process, Figure 1 depicts a Redis operator incident caused by missing TLS client certificate arguments [26]. When Redis requires client certificates, the operator repeatedly fails cluster health checks, preventing the `RedisCluster` resource from reaching the `Ready` state despite healthy server Pods. During detection, the agent inspects runtime cluster observations to identify this operational stall. During root-cause analysis, it combines operator logs with client probes to trace the TLS authentication failure to missing `--cert` and `--key` arguments in `redis-cli` invocations. During remediation, the agent modifies the controller implementation, while the harness builds and deploys the repair and returns runtime feedback. Recovery is then verified through fault clearance and functional checks. This case underscores that resolving realistic cloud incidents requires traversing the complete lifecycle rather than inspecting code or telemetry in isolation.

Existing benchmarks evaluate isolated segments of the incident resolution lifecycle, as summarized in Table 1. Cloud operations benchmarks evaluate anomaly detection and operational

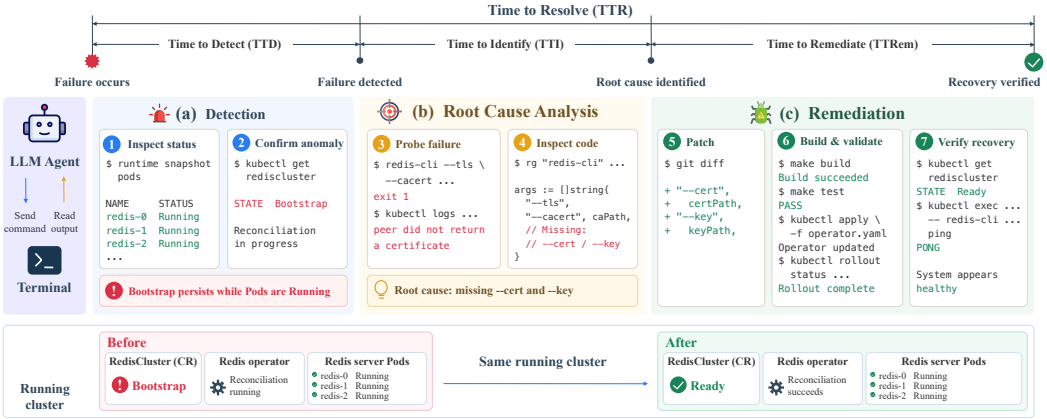


Fig. 1. An example incident resolution lifecycle, spanning Time to Detect (TTD), Time to Identify (TTI), and Time to Remediate (TTRem) toward the overall Time to Resolve (TTR).

mitigation, predominantly relying on synthetic fault injection in microservices [6, 14, 28, 31, 36]. However, they do not require agents to repair historical software defects in their original repositories or validate deployed recovery. Conversely, software engineering benchmarks evaluate program repair from static issue descriptions against offline test suites [15, 16, 38]. These protocols omit runtime incident discovery in live clusters and operational recovery verification. Consequently, existing benchmarks leave open whether autonomous agents can navigate the complete incident resolution lifecycle to achieve verified recovery.

Addressing these limitations requires an evaluation setting grounded in confirmed historical defects and live operational execution. Reproducing incidents within their original repositories preserves authentic failure modes, triggering conditions, and native build workflows. In real-world cloud environments, resolution efficiency is paramount: delayed anomaly detection and failed repair attempts directly prolong service disruptions [9, 18]. Evaluation must therefore treat time not merely as an operational timeout, but as an intrinsic performance objective across the lifecycle [3, 23]. This requires assessing verified recovery alongside the operational latency (*e.g.*, Time to Identify (TTI) and Time to Resolve (TTR)) and resource expenditures necessary to achieve it.

To realize these requirements, we introduce SWEOps-Bench, an interactive benchmark for evaluating agents on end-to-end resolution of software-induced cloud incidents. SWEOps-Bench contains 55 incident tasks reproduced from confirmed historical defects in 16 open-source Kubernetes operator projects managing databases, stream processors, and infrastructure services. Each task initializes in a healthy cluster environment to evaluate false-positive resistance before an automated trigger induces the target incident. Agents interact with the live cluster and source repository through a bash terminal, while the benchmark harness builds and deploys candidate repairs to return runtime execution feedback. Across this lifecycle, SWEOps-Bench evaluates both the effectiveness and efficiency of autonomous agents in achieving verified recovery.

We evaluate terminal-based agent harnesses, including Codex and Claude Code, across four recent proprietary and open-weight models spanning GPT-5.6 Luna, Claude-Sonnet-5, DeepSeek-V4.1-Flash, and GLM-5.3-Flash. Our evaluation characterizes incident resolution effectiveness alongside lifecycle latency and token consumption, revealing three central empirical findings. First, end-to-end resolution remains substantially harder than incident recognition: faulty-state detection ranges from 72.7% to 90.9%, while overall recovery ranges from 14.5% to 52.7% across the evaluated configurations. Second, relative advantages shift across lifecycle stages and agent–model configurations, and

substantially greater token consumption does not consistently translate into higher recovery. Third, diagnosis and remediation are only partially aligned: for the two representative configurations examined at the trajectory level, 14.5%–21.8% of tasks achieve verified recovery despite an incorrect reported joint RCA, while runtime symptom clearance can still violate fix-specific functional requirements.

Our principal contributions are summarized as follows:

- **Realistic Benchmark.** We construct SWEOps-Bench with 55 incident tasks derived from confirmed historical defects in 16 open-source projects. Each task preserves the original repository context and reproduces the failure in an isolated cluster environment.
- **Evaluation Framework.** We formulate end-to-end incident resolution as an interactive task spanning runtime detection, source code-level diagnosis, and deployed source repair. The framework combines curated ground truth with executable recovery checks to evaluate resolution effectiveness and efficiency.
- **Empirical study.** We evaluate frontier terminal-based agent harnesses and models on SWEOps-Bench, characterizing resolution capabilities, stage bottlenecks, and resource expenditures.

2 Background and Motivation

2.1 Cloud Application Management

Kubernetes operators automate application management by combining custom resources with dedicated software controllers [10, 11]. Operators encode domain-specific operational expertise into software and interact with the platform API server to manage application workloads. These controllers execute control loops governed by the state-reconciliation principle: they continuously observe the actual state of managed resources, compare it with the desired state declared in the custom specification, and issue operations to drive the system toward agreement [30].

The reliability of management software is critical because defects in operator logic affect the stability of the managed applications (*e.g.*, by generating invalid resource specifications or preventing managed applications from reaching their desired states) [10, 11]. For example, a defect in a Redis operator causes the generated Pod specification to include an init container with a volume mount that references an undefined volume when persistence is disabled [29]. The Kubernetes API server rejects the resulting Pod specification, preventing the cluster from completing initialization. The failure manifests in runtime resources, whereas the underlying defect resides in the operator implementation. Resolving such incidents therefore requires connecting runtime symptoms to the responsible controller source code, repairing the defect, and verifying the recovery.

2.2 Cloud Incident Resolution Lifecycle

In production environments, resolving software-induced incidents is not an isolated, one-off action, but an operational lifecycle governed by dependability and Site Reliability Engineering (SRE) principles [1–3]. Prior empirical studies show that cloud incident management spans continuous phases of anomaly detection, diagnostic investigation, and mitigation [9, 18]. In practice, these activities form an interdependent feedback loop: diagnostic hypotheses guide candidate repairs, while runtime execution feedback from deployed changes informs further investigation and code revision. Because prolonged disruptions severely degrade service availability, industrial practice treats resolution latency as a primary operational metric alongside functional restoration [23].

As illustrated in Figure 1, SWEOps-Bench models this resolution lifecycle across three foundational stages:

- **Detection.** Detection concerns recognizing abnormal system behavior from runtime observations. In cloud-native environments, resource states, platform events, and controller logs provide

Table 1. Comparison of SWEOps-Bench with related benchmarks.

Benchmark	Target Systems	Task Construction	Detect Eval	RCA Eval	Source Repair	Runtime Recovery	Full Lifecycle
OpsEval [22]	Curated scenarios	Case questions	✗	✓*	✗	✗	✗
AIOpsLab [6]	Demo microservices	Fault injection	✓	✓	✗	✓	✗
OpenRCA [36]	Enterprise telemetry	Failure records	✗	✓	✗	✗	✗
ITBench (SRE) [14]	Demo microservices	Fault injection	✗	✓	✗	✓	✗
SREGym [8]	Demo microservices	Fault injection	✗	✓	✗	✓	✗
Cloud-OpsBench [31]	Demo microservices	Fault injection	✗	✓	✗	✗	✗
RCAEval [28]	Demo microservices	Fault injection	✗	✓	✗	✗	✗
SWE-bench [15]	Open-source repos	Curated issues	✗	✗	✓	✗	✗
SWEOps-Bench	**Open-source operators**	**Historical defects**	✓	✓	✓	✓	✓

Notes: *OpsEval assesses diagnostic reasoning through curated Q&A.

Full Lifecycle connects detection, diagnosis, source repair, and recovery verification within one evaluation protocol, measuring elapsed time from incident onset to the recovery declaration for subsequently verified tasks.

complementary evidence, as individual resource status does not necessarily establish overall application health.

- **Root Cause Analysis.** RCA traces an observed runtime failure to its underlying cause. For software-induced incidents, source-level localization identifies the files and functions, or configuration files, responsible for the failure.
- **Remediation.** Remediation addresses the underlying defect to restore required system functionality. For implementation bugs, this involves modifying and redeploying the affected software, followed by executable verification confirming both failure elimination and preservation of existing capabilities.

Together, these stages capture the progression from incident onset to verified recovery. We characterize lifecycle efficiency across this progression using Time to Detect (TTD), Time to Identify (TTI), Time to Remediate (TTRem), and overall Time to Resolve (TTR), whose formal boundaries are defined in Section 3.5.

2.3 Prior Benchmarks and Motivation

As autonomous agents increasingly bridge system operations and code modification, incident handling shifts from fragmented tasks toward unified workflows. However, as summarized in Table 1, existing benchmarks evaluate isolated segments of this lifecycle under disparate formulations. Cloud operations benchmarks assess diagnostic reasoning and operational mitigation, typically relying on synthetic fault injection in microservices [6, 8, 14, 31] or offline telemetry snapshots [22, 28, 36, 41]. However, they do not require agents to resolve defects in the original controller codebase or validate recovery after redeployment. Conversely, software engineering benchmarks evaluate repository-level program repair from static issue descriptions [15, 38], omitting runtime incident discovery and deployed operational validation.

To evaluate autonomous agents across the complete incident resolution lifecycle, the research community requires an interactive benchmark grounded in confirmed historical defects reproduced within their original repositories. Reproducing authentic defects captures complex reconciliation failures while providing developer patches and regression suites as objective ground truth. These considerations motivate four essential design requirements for lifecycle evaluation:

- **R1: Runtime incident discovery.** Evaluation requires agents to detect anomalies directly from live cluster observations without pre-packaged issue descriptions. Agents independently discriminate healthy baseline states from emerging failures before initiating diagnostic investigation.

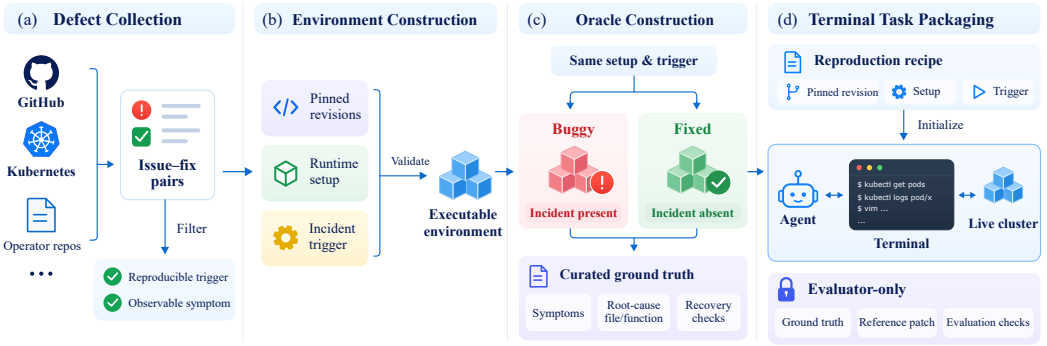


Fig. 2. Overview of the SWEOps-Bench task construction pipeline.

- **R2: Deployable source repair.** Resolving software-induced incidents requires correcting the defective implementation rather than applying temporary operational workarounds. Candidate repairs should integrate into the native build workflows and deploy to the live cluster, enabling feedback-driven iterative revision.
- **R3: Verified functional recovery.** Symptom disappearance alone does not establish resolution, as superficial patches or pod restarts can mask failures while introducing regressions. Verification combines case-specific fault oracles with end-to-end management checks to confirm that the defect is eliminated and required functionality is preserved.
- **R4: End-to-End lifecycle evaluation.** Rather than evaluating isolated sub-tasks, evaluation spans detection, diagnosis, source repair, and recovery verification under standardized operational budgets. This unified protocol exposes how agents backtrack from deployment failures and the overall latency and token costs incurred.

3 SWEOps-Bench Construction

To realize the design requirements established in Section 2.3, we present SWEOps-Bench, an interactive benchmark for end-to-end autonomous incident resolution. Figure 2 summarizes its construction pipeline, from historical defect collection and environment reconstruction to oracle construction and terminal-task packaging. Sections 3.2.1–3.5 detail the dataset construction, evaluation oracles, task execution protocol, and metrics for resolution effectiveness, lifecycle efficiency, and token consumption.

3.1 Task Formulation

SWEOps-Bench formulates incident resolution as an interactive task over a live Kubernetes environment and its management software repository. The agent receives runtime observations and terminal access, and produces anomaly decisions, a root-cause report, source or configuration changes, and a recovery declaration.

3.2 Dataset Construction

SWEOps-Bench contains 55 incident tasks constructed from confirmed historical defects in real-world Kubernetes management software. Below, we detail our candidate defect collection, two-phase screening methodology, and execution environment reconstruction.

3.2.1 Defect Collection and Selection. Following controller reliability studies [10, 11], we select operator projects based on project maturity, source code availability, and application diversity. As summarized in Table 2, the benchmark encompasses 16 representative operator projects across

Table 2. Upstream operator projects and incident distributions in SWEOps-Bench.

Operator	Managed System	Stars	KLOC	Cases	Operator	Managed System	Stars	KLOC	Cases
RedisOp	Redis	1,464	10.1	9	ZKOp	ZooKeeper	383	2.5	3
VMOp	VictoriaMetrics	590	50.6	9	GrafanaOp	Grafana	1,389	11.5	2
MongoOp	MongoDB	1,357	10.1	5	SparkOp	Apache Spark	3,154	11.4	2
PCN/MongoOp	MongoDB	457	27.9	5	OTelOp	OpenTelemetry	1,762	28.4	2
CNPG	PostgreSQL	9,374	62.4	4	Rook	Ceph	13,670	61.1	2
CassOp	Cassandra	228	13.1	3	LoggingOp	Fluent Bit	1,715	19.6	1
KnativeOp	Knative	225	6.0	3	PromOp	Prometheus	9,983	33.7	1
PCN/XtraDBOp	Percona XtraDB	625	25.8	3	RabbitMQOp	RabbitMQ	1,117	5.8	1
Total: 16 operator repositories					**55 historical incident tasks**				

Notes: Operator names provide clickable hyperlinks directly to their upstream GitHub repositories. Stars and KLOC were measured on September 29, 2026.

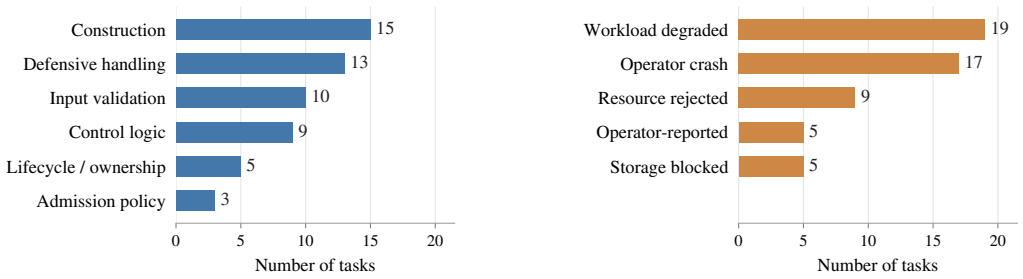
diverse functional domains, including distributed databases, stream processing, messaging, observability, and serverless. All repositories are officially developed and maintained by recognized open-source communities (*e.g.*, CNCF) or reputable commercial software vendors (*e.g.*, MongoDB).

We collect candidate defects from prior controller reliability studies and GitHub issue histories. The former include defects documented by Sieve [30], Acto [10], and Oat [11]. To reflect modern cloud-native operational challenges and reduce the potential pre-training exposure window, the second source mines contemporary bug fixes merged between January and August 2026 across 16 repositories, inspecting maintainer-confirmed pull requests with bug-related tags on the main development branch.

We screen candidates using three exclusion criteria: (E1) *Unverified fix*: no verifiable merged upstream fix is available, or the referenced fix was reverted; (E2) *Out-of-scope defect*: the change is confined to documentation, comments, third-party deployment tooling, or the managed application rather than the operator implementation or operator-owned configuration; (E3) *Duplication or reconstruction constraints*: the report duplicates a defect already represented by the same upstream fix, or reproduction requires unavailable proprietary services or infrastructure. Candidates that pass this screening proceed to environment reconstruction and execution-based validation.

3.2.2 Environment Reconstruction and Validation. For each candidate defect passing initial screening, we reconstruct an isolated runtime environment using the original management software (Figure 2(b)). We pin the affected source revision, Kubernetes release, container images, dependencies, and deployment manifests used to reproduce the target failure. The environment must reach a healthy initial state in which the operator and managed resources satisfy the required health checks. A case-specific triggering procedure then exposes the existing defect without modifying the target implementation.

We validate each reconstructed case through paired executions of the buggy and upstream-fixed revisions under the same environment and triggering procedure. This execution-based screening excludes candidates that meet any of the following criteria: (E4) *Bootstrapping failure*: the cluster, operator, or managed resources cannot reach the required healthy initial state; (E5) *Unobservable failure*: the defect has no verifiable runtime manifestation in Kubernetes resources, management operations, or managed workloads; (E6) *Unreproducible environment or trigger*: required dependencies cannot be obtained, or the triggering procedure cannot reliably reproduce the target failure. A task is retained only if the buggy revision reproduces the target incident and the upstream-fixed revision eliminates the failure while passing the required functional and regression checks. This



(a) Fault mechanisms (b) Runtime manifestations

Fig. 3. Distribution of 55 incident tasks by (a) primary fault mechanism and (b) dominant runtime manifestation. Bar-end labels indicate task counts.

two-phase screening process yields 55 verified incident tasks: 28 from prior controller reliability studies and 27 from the additional 2026 collection.

Figure 3 summarizes the distribution of the final 55 incident tasks. Across defect mechanisms (Figure 3(a)), the tasks are grouped into six categories: Construction (15 cases), Defensive Handling (13), Input Validation (10), Control Logic (9), Lifecycle / Ownership (5), and Admission Policy (3). Across runtime manifestations (Figure 3(b)), the tasks are grouped into five categories: Workload degraded (19 cases), Operator crash (17), Resource rejected (9), Operator-reported (5), and Storage blocked (5).

3.3 Evaluation Oracle Construction

To evaluate autonomous agents across the complete lifecycle, SWEOps-Bench constructs dedicated evaluation oracles for each operational stage (Figure 2(c)).

For runtime detection, we construct an executable fault oracle implemented as a case-specific probe derived from the differential behavior between buggy and fixed revisions. Depending on the failure mechanism, the probe checks Kubernetes resource conditions, controller event logs, or workload service endpoints for evidence of the target incident. Observations are labeled as normal when the system satisfies the required health checks and as abnormal when the fault oracle detects the target failure. The earliest confirmed manifestation establishes the reference timestamp for measuring Time to Detect (TTD).

Reference root-cause annotations are derived from maintainer-confirmed issue reports and merged upstream fixes. We inspect the affected implementation and reproduced runtime behavior to verify the defect mechanism and identify the responsible source or configuration files and, where applicable, functions. Auxiliary changes in the fixing commit are not automatically treated as root-cause locations.

For remediation, each task incorporates executable checks for build and deployment success, target fault elimination, and required functional behavior. The fault oracle determines whether the target failure remains observable in the running cluster. To verify genuine recovery, fix-specific functional tests exercise the affected capability under triggering conditions, while regression suites check selected previously working behavior. These test oracles are constructed by adapting upstream PR regression tests and project integration suites, confirmed by differential execution across the buggy and fixed revisions. Crucially, these checks evaluate the runtime behavior of the repaired implementation rather than textual similarity to the upstream developer patch, accommodating alternative valid repairs while ensuring symptom elimination does not come at the expense of required functionality.

3.4 Task Packaging and Execution

Each incident task is packaged into an execution bundle comprising the environment specification, pinned operator repository, automated triggering workload, and hidden evaluation oracles (Figure 2(d)). To restrict access to external task information, outbound internet access is disabled within the agent's tool execution environment, restricting network connectivity strictly to the local Kubernetes cluster. The original issue discussions, reference fixes, and verification scripts remain under benchmark control and are withheld from the agent throughout execution.

The benchmark harness equips the agent with an interactive bash terminal providing direct command-line access to the live cluster via `kubectl` and Unix utilities, alongside read-write access to the local operator repository. Agents use terminal commands to inspect system behavior, search and modify source files. When a candidate repair is submitted, the harness compiles the source code, rebuilds the controller image, deploys the updated controller into the cluster, and returns execution feedback, such as compiler errors or rollout failures. This feedback loop enables iterative revision, with file modifications, runtime logs, and cluster state changes carrying forward continuously within the interactive session.

Each evaluation session proceeds through a standardized execution protocol. The harness first validates cluster readiness and presents the agent with an initial observation from a healthy baseline state, serving as a negative control to evaluate false-positive detection decisions. The healthy-state phase operates under a separate wall-clock budget of $B = 60$ minutes. The harness then executes the case-specific triggering workload, and a benchmark-controlled fault probe establishes when the target incident becomes observable. Once the incident manifests, the fault-state phase starts with a fresh conversation context and operates under an independent wall-clock budget of $B = 60$ minutes covering all subsequent investigation, diagnosis, code modification, and deployment attempts. The fault-state phase ends when the agent explicitly declares recovery in the terminal or when the operational budget expires. The harness then applies the hidden repair and recovery checks to the submitted changes and resulting system state.

3.5 Evaluation Methodology and Metrics

SWEOps-Bench evaluates autonomous incident resolution across stage-wise decision quality, verified remediation, lifecycle latency, and resource costs. Evaluation builds directly upon the ground-truth annotations and executable oracles established in Section 3.3. Agent implementations may internally invoke specialized subagents for detection, RCA, and remediation; such delegation is treated as an internal orchestration mechanism and does not alter the evaluation protocol or metrics.

To evaluate anomaly detection, we assess whether agents reliably discriminate healthy operational states from active incidents. *Healthy* measures correct recognition during the initial negative-control baseline, and *Faulty* measures correct anomaly identification once the target incident manifests. We report *Paired* detection, which strictly requires both decisions to be correct within the same task execution. This paired metric explicitly penalizes trivial heuristics that indiscriminately report perpetual normality or continuous anomalies.

We evaluate RCA at the *file* and *function* levels and additionally report *joint accuracy*, which requires both locations to be correct. For configuration-induced incidents where function-level ground truth does not apply, joint correctness evaluates whether the causal configuration files are accurately identified. Detection and diagnosis outcomes are reported independently from final system recovery, enabling empirical characterization of instances where agents achieve verified recovery despite incomplete or incorrect diagnostic reports.

To dissect remediation, Table 4 reports pass rates for seven outcomes: code compilation (*Build*), container rollout (*Deploy*), a non-empty, parseable patch modifying a ground-truth root-cause file (*Patch Check*), fault clearance confirmed by the fault probe (*Fault Cleared*), regression testing (*Regression Tests*), and verification of affected capabilities under triggering conditions (*Fix-Specific Tests*), and *Agent-Declared Recovery*, which indicates whether the agent explicitly declares recovery within the execution budget. *Patch Check* is auxiliary and not a recovery criterion, since a valid repair may modify alternative implementation locations. Pass rates are computed over applicable tasks, counting unattempted or failed checks as unsuccessful. Overall *Recovery* requires an explicit recovery declaration within the execution budget and success on all applicable criteria except *Patch Check*.

To evaluate lifecycle efficiency, we measure Time to Detect (*TTD*), Time to Identify (*TTI*), Time to Remediate (*TTRem*), and overall Time to Resolve (*TTR*) in minutes. Reporting latency only for successful executions can obscure differences in completion rates and in the sets of tasks solved. For task i, detection duration $T_{i,\mathrm{D}}$ is measured from incident manifestation to the first correct anomaly report. Identification duration $T_{i,\mathrm{I}}$ is measured from the first correct anomaly report to the first correct root-cause report. Remediation duration $T_{i,\mathrm{Rem}}$ is measured from the first correct root-cause report to the recovery declaration, provided that recovery is subsequently verified. A stage duration is defined only when both boundary milestones occur in the specified order. These durations measure intervals between reporting milestones rather than time spent exclusively on each activity. Overall resolution time is measured from incident manifestation to the recovery declaration for tasks that subsequently pass recovery verification. It remains defined even when an intermediate reporting milestone is missing.

Inspired by penalized runtime evaluation [13], we retain observed durations for achieved milestones and assign predetermined penalties otherwise. An unachieved detection, identification, or remediation milestone receives a penalty of B, $3B$, or $6B$ minutes, respectively. Let $\widetilde{T}_{i,\mathrm{D}}$, $\widetilde{T}_{i,\mathrm{I}}$, and $\widetilde{T}_{i,\mathrm{Rem}}$ denote these penalized stage latencies.

For the overall resolution, we retain the observed end-to-end TTR, measured to the agent's recovery declaration, when task i subsequently passes recovery verification. Otherwise, we aggregate the penalized stage latencies:

$$\widetilde{\mathrm{TTR}}_i = \begin{cases} T_{i,\mathrm{TTR}}, & \text{if verified recovery is achieved,} \\ \widetilde{T}_{i,\mathrm{D}} + \widetilde{T}_{i,\mathrm{I}} + \widetilde{T}_{i,\mathrm{Rem}}, & \text{otherwise.} \end{cases} \tag{1}$$

Thus, a successfully recovered task retains its observed wall-clock resolution time even if an intermediate reporting milestone is missing, whereas unresolved tasks are penalized according to the lifecycle stages they fail to complete. A task achieving none of the milestones receives the maximum combined penalty of $10B$ minutes. These penalized quantities are evaluation metrics expressed in time units and do not extend the session budget. We report mean penalized TTD, TTI, TTRem, and TTR across all 55 benchmark tasks.

Finally, we measure operational overhead by reporting mean input and output token counts per trajectory separately for healthy-state and fault-state execution. Input counts include cached tokens, and we additionally report the cached-input proportion (*Cache (%)*).

4 Empirical Evaluation

We evaluate LLM-based agents on SWEOps-Bench through four research questions on lifecycle effectiveness, efficiency, robustness, and trajectory behavior:

RQ1 (Lifecycle Effectiveness): How effectively do agents perform at individual lifecycle stages and achieve verified incident resolution?

Table 3. Overall performance of evaluated agent–model configurations on SWEOps-Bench.

Agent	Model	Detection (%)			RCA (%)			REC (%)
		Healthy	Faulty	Paired	File	Func.	Joint	
Codex	GLM-5.3-Flash	72.7	74.5	60.0	54.5	43.6	43.6	34.5
	DeepSeek-V4.1-Flash	43.6	85.5	38.2	63.6	**50.9**	**50.9**	45.5
	GPT-5.6 Luna	85.5	76.4	72.7	50.9	36.4	36.4	30.9
Claude Code	GLM-5.3-Flash	67.3	72.7	45.5	25.5	5.5	5.5	14.5
	DeepSeek-V4.1-Flash	83.6	87.3	72.7	**65.5**	30.9	30.9	47.3
	Claude-Sonnet-5	**94.5**	**90.9**	**85.5**	**65.5**	38.2	38.2	**52.7**

RQ2 (Lifecycle Efficiency): What time and token costs do agents incur across the incident resolution lifecycle?

RQ3 (Robustness): How robust is agent resolution effectiveness across incident characteristics?

RQ4 (Trajectory Analysis): What trajectory behaviors shape incident resolution outcomes?

4.1 Experimental Setup

All evaluations follow the protocol and metrics defined in Sections 3.4–3.5.

4.1.1 Agents and Models. We evaluate two terminal-based agent harnesses: Codex and Claude Code. We evaluate four models spanning proprietary and open-weight models available during the evaluation period. Specifically, our proprietary suite comprises OpenAI GPT-5.6 Luna (`gpt-5.6-luna`, 2026-07-09) and Anthropic Claude-Sonnet-5 (`claude-sonnet-5`, 2026-06-30). Our open-weight suite comprises DeepSeek-V4.1-Flash (2026-09-10) and Zhipu GLM-5.3-Flash (`glm-5.3-flash`, 2026-08-26). All evaluations directly query the official first-party API endpoints provided by the respective model vendors between August and September 2026, avoiding third-party proxies. We employ the official default generation parameters and reasoning-effort configurations across all models without bespoke hyperparameter tuning.

4.1.2 Execution Infrastructure. We use the Harbor benchmarking framework [12] to orchestrate and manage containerized task environments across SWEOps-Bench. For each incident task, Harbor provisions an isolated Kubernetes cluster along with the corresponding management operator and runtime dependencies. Each cluster operates with pinned Kubernetes releases matching the historical defect requirements, drawn from four versions: v1.21.14+k3s1, v1.23.17+k3s1, v1.28.10+k3s1, and v1.31.5+k3s1. All task environments run on a single virtual machine provisioned through OpenStack Nova with KVM virtualization, configured with 16 vCPUs, 64 GiB of memory, and Ubuntu 24.04.4 LTS (x86_64). Each task environment is capped at 2 vCPUs and 7 GiB of memory, with the memory limit increased to 10–12 GiB for four memory-intensive cases. To limit resource contention, we run at most four task environments concurrently.

We conduct one independent run per task and agent–model configuration. Harbor captures full interaction trajectories, shell command logs, runtime resource snapshots, and token consumption metrics for post-execution analysis.

4.2 RQ1: Lifecycle Effectiveness

Table 3 reports effectiveness across detection, RCA, and overall recovery (REC). Agents generally recognize active incidents more reliably than they discriminate between healthy and faulty states. Faulty-state detection ranges from 72.7% to 90.9%, while paired detection ranges more widely from 38.2% to 85.5%. The largest gap appears for Codex + DeepSeek-V4.1-Flash, which achieves 85.5%

Table 4. Detailed remediation outcomes of evaluated agent–model configurations. All results are reported as percentages. Component rates use applicable-task denominators.

Agent	Model	Build	Deploy	Patch Check	Fault Cleared	Regression Tests	Fix-Specific Tests	Agent-Declared Recovery
Codex	GLM-5.3-Flash	100	63.6	56.4	50.9	100	29.4	47.3
	DeepSeek-V4.1-Flash	100	**89.1**	72.7	63.6	100	47.1	60.0
	GPT-5.6 Luna	100	76.4	58.2	54.5	100	35.3	47.3
Claude Code	GLM-5.3-Flash	100	29.1	27.3	25.5	97.9	35.3	20.0
	DeepSeek-V4.1-Flash	100	87.3	**80.0**	**72.7**	97.9	41.2	60.0
	Claude-Sonnet-5	100	81.8	72.7	61.8	97.9	**58.8**	72.7

faulty-state detection but only 38.2% paired accuracy. By contrast, Claude Code + Claude-Sonnet-5 achieves the highest paired detection rate of 85.5%. These results demonstrate the importance of evaluating healthy and faulty states jointly: high anomaly recognition alone does not imply reliable incident detection.

Performance becomes more differentiated from detection to source-level diagnosis and recovery. File-level RCA ranges from 25.5% to 65.5%, while function-level and joint RCA range from 5.5% to 50.9%. All configurations show lower function-level than file-level accuracy, indicating that identifying the relevant file is more reliable than pinpointing the responsible implementation. Relative model performance also changes across lifecycle stages. Under Codex, GPT-5.6 Luna achieves the highest paired detection, whereas DeepSeek-V4.1-Flash achieves the highest joint RCA and recovery. Under Claude Code, Claude-Sonnet-5 leads paired detection, joint RCA, and recovery. Thus, stronger detection or RCA does not consistently translate into stronger end-to-end resolution.

The harness comparison shows a similarly non-uniform pattern for the two models evaluated with both agents. With GLM-5.3-Flash, Codex performs better in paired detection, joint RCA, and recovery. With DeepSeek-V4.1-Flash, Claude Code achieves higher paired detection and recovery, while Codex achieves higher joint RCA. These contrasting results indicate that neither harness provides a uniform advantage across models and lifecycle stages. Agent effectiveness should therefore be interpreted at the agent–model configuration level rather than attributed to the model or harness in isolation.

Table 4 provides a fine-grained view of the remediation process, from build and deployment to runtime fault clearance, functional verification, and agent-declared recovery. Build succeeds on 100% of applicable tasks, and broad regression-test pass rates remain between 97.9% and 100%, while deployment and fix-specific verification are substantially less reliable. Runtime symptom clearance does not establish semantic repair correctness: a modification can remove the immediate failure while changing how the operator handles supported requests or invalid inputs. Fix-specific verification examines these management requirements beyond the disappearance of the triggering symptom.

4.3 RQ2: Lifecycle Efficiency

Table 5 reports penalized lifecycle latencies. Penalized latencies increase substantially toward later stages, with TTD consistently lower than TTI and TTRem, indicating that lifecycle cost is concentrated after incident recognition.

Penalized TTR broadly tracks recovery performance, although configurations with comparable TTR can still exhibit different recovery rates. For example, Claude Code + Claude-Sonnet-5 and

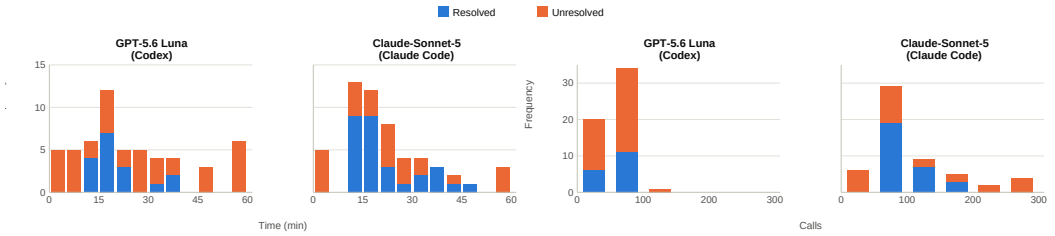


Fig. 4. Distributions of agent execution time and number of model calls.

Table 5. Penalized lifecycle latencies of evaluated agent–model configurations (min), averaged over all 55 tasks.

Agent	Model	$\widetilde{\text{TTD}}$	$\widetilde{\text{TTI}}$	$\widetilde{\text{TTRem}}$	$\widetilde{\text{TTR}}$
Codex	GLM-5.3-Flash	21.2	103.5	239.6	351.7
	DeepSeek-V4.1-Flash	17.4	96.2	208.0	285.7
	GPT-5.6 Luna	19.7	117.4	252.3	361.9
Claude Code	GLM-5.3-Flash	23.1	171.1	309.4	477.4
	DeepSeek-V4.1-Flash	13.3	129.1	194.3	269.8
	Claude-Sonnet-5	10.9	116.1	177.3	263.7

Claude Code + DeepSeek-V4.1-Flash achieve penalized TTRs of 263.7 and 269.8 minutes, with recovery rates of 52.7% and 47.3%, respectively. Figure 4 further shows that unresolved trajectories can persist despite extended execution and many model calls. Thus, penalized TTR is informative but not a one-to-one proxy for recovery, while greater execution effort does not uniformly translate into successful recovery.

Fault-state token consumption differs substantially across configurations (Table **??**). For the two models evaluated with both harnesses, Claude Code uses 4.1–4.5× more input tokens and 6.6–9.6× more output tokens than Codex, yet recovery decreases for GLM-5.3-Flash (34.5% to 14.5%) and increases only slightly for DeepSeek-V4.1-Flash (45.5% to 47.3%). Thus, greater token use does not by itself imply greater end-to-end effectiveness.

4.4 RQ3: Robustness across Incident Characteristics

For focused analysis, we select one vendor-native proprietary configuration for each agent harness: Codex + GPT-5.6 Luna and Claude Code + Claude-Sonnet-5. Figure 5 examines whether agent effectiveness is robust across variations in defect mechanism, runtime manifestation, and issue recency.

Recovery varies substantially across defect mechanisms and runtime manifestations (Figure 5(a–b)). Input Validation constitutes a shared weakness, with recovery rates of 40.0% for Claude Code + Claude-Sonnet-5 and 10.0% for Codex + GPT-5.6 Luna. These tasks require agents to distinguish supported from invalid inputs. Normalization or fallback handling can resolve valid cases but may also suppress validation errors for inputs that should instead be rejected. For example, one agent normalized an invalid version value and incorrectly treated the resulting error clearance as success.

Beyond this common weakness, the configurations exhibit different patterns across categories. Claude Code + Claude-Sonnet-5 achieves higher recovery in Defensive Handling (61.5% vs. 38.5%) and Operator crash (70.6% vs. 35.3%), whereas Codex + GPT-5.6 Luna achieves higher recovery in Storage blocked (60.0% vs. 40.0%). The two configurations tie on Lifecycle / Ownership (40.0%).

Table 6. Token consumption of evaluated agent–model configurations, separated by healthy-state and fault-triggered execution. Values are means per trajectory. Cached tokens are included in input tokens.

		Healthy State			Fault State		
Agent	Model	Input	Output	Cache (%)	Input	Output	Cache (%)
Codex	GLM-5.3-Flash	292.8K	5.1K	82.9	910.4K	15.2K	89.2
	DeepSeek-V4.1-Flash	3127.5K	24.0K	98.5	1902.2K	21.5K	97.8
	GPT-5.6 Luna	654.8K	7.4K	86.8	1845.7K	19.8K	88.8
Claude Code	GLM-5.3-Flash	853.5K	23.9K	83.4	3703.3K	100.8K	79.8
	DeepSeek-V4.1-Flash	2536.8K	66.3K	95.6	8500.4K	206.8K	96.1
	Claude-Sonnet-5	669.4K	7.5K	78.7	4620.6K	44.1K	75.9

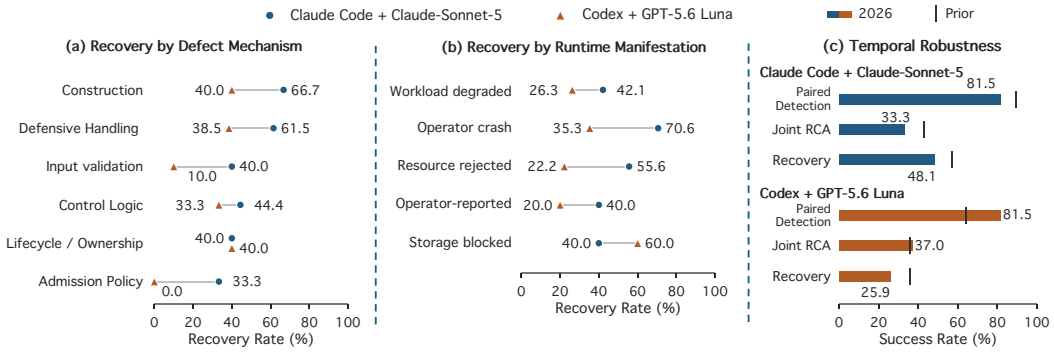


Fig. 5. Agent effectiveness across incident characteristics: (a) recovery by defect mechanism, (b) recovery by runtime manifestation, and (c) lifecycle-stage performance by issue recency.

These differences show that aggregate recovery rates mask substantial variation across incident characteristics and agent configurations.

We further examine performance by issue recency because older incidents have had more opportunity to appear in publicly available data. Figure 5(c) compares the 28 incidents from prior studies with the 27 collected in 2026. Claude Code + Claude-Sonnet-5 performs better on prior incidents across all three metrics, whereas Codex + GPT-5.6 Luna performs better on 2026 incidents for paired detection and joint RCA but worse for recovery. The largest recency gap is 17.2 percentage points in paired detection for Codex + GPT-5.6 Luna, while Claude Code + Claude-Sonnet-5 shows smaller differences across the three metrics. Overall, the results show no uniform degradation on newly collected incidents.

4.5 RQ4: Trajectory Behaviors and Resolution Outcomes

To understand how trajectory-level behaviors shape incident resolution outcomes, we further analyze these two representative configurations. We first manually inspect failed trajectories using a taxonomy adapted from the trajectory analysis of Terminal-Bench [24]. Two annotators calibrated and refined the category definitions on a subset of trajectories and then annotated the failed trajectories, resolving disagreements through discussion. The taxonomy covers execution, coherence, adaptation, assessment, and control. Annotations are multi-label, so a trajectory may exhibit multiple failure modes. Figure 6 compares the prevalence of these modes across the two configurations. We then examine when verified recovery occurs across repair attempts and how

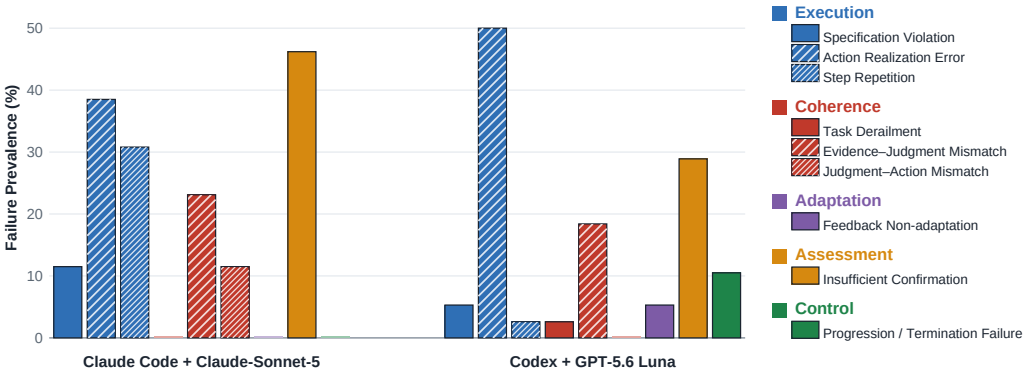


Fig. 6. Prevalence of behavioral failure modes in failed trajectories across the two agent configurations.

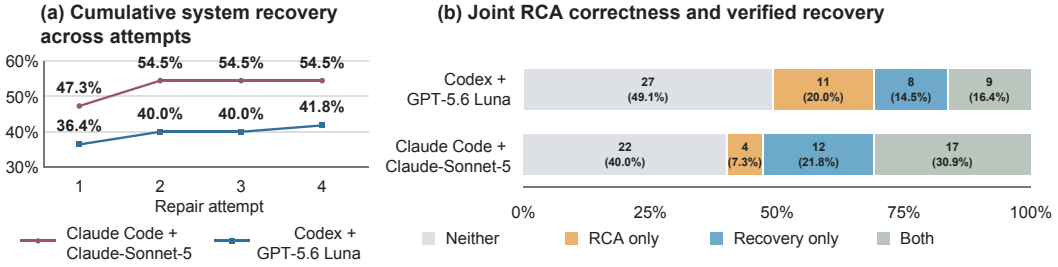


Fig. 7. (a) Cumulative system-side recovery across repair attempts; (b) relationship between joint RCA correctness and verified recovery.

reported RCA correctness corresponds to final verified recovery, before using representative trajectories to illustrate how these patterns arise during execution.

As shown in Figure 6, failures in both configurations are concentrated in Action Realization Error (ARE) and Insufficient Confirmation (IC). For Claude Code + Claude-Sonnet-5, IC occurs in 46.2% of failed trajectories and ARE in 38.5%; for Codex + GPT-5.6 Luna, the corresponding rates are 28.9% and 50.0%. Claude Code additionally exhibits substantial Step Repetition (30.8%), whereas Codex shows more Evidence–Judgment Mismatch (EJM, 18.4%). Other modes are less frequent. Judgment–Action Mismatch (JAM) occurs only in Claude Code (11.5%), while Task Derailment, Feedback Non-adaptation, and Progression / Termination Failure are absent for Claude Code + Claude-Sonnet-5 and remain at or below 10.5% for Codex + GPT-5.6 Luna.

Action Realization Error often reflects incomplete implementation of a plausible intervention, such as modifying only some locations implicated in a defect. Insufficient Confirmation and Evidence–Judgment Mismatch instead arise when agents observe residual anomalies but dismiss them on insufficient evidence. The key issue is therefore not whether an anomaly is observed, but whether its exclusion from the resolution target is justified.

Figure 7(a) reports cumulative system-side recovery across successive repair attempts, independent of whether the agent declares recovery. Claude Code + Claude-Sonnet-5 recovers 47.3% of tasks within one repair attempt and 54.5% within two, with no additional recoveries from later attempts. Codex + GPT-5.6 Luna recovers 36.4% within one attempt, 40.0% within two, and reaches 41.8% within four. Thus, additional repair attempts can recover cases that remain unresolved after the initial intervention, although most observed gains occur early.

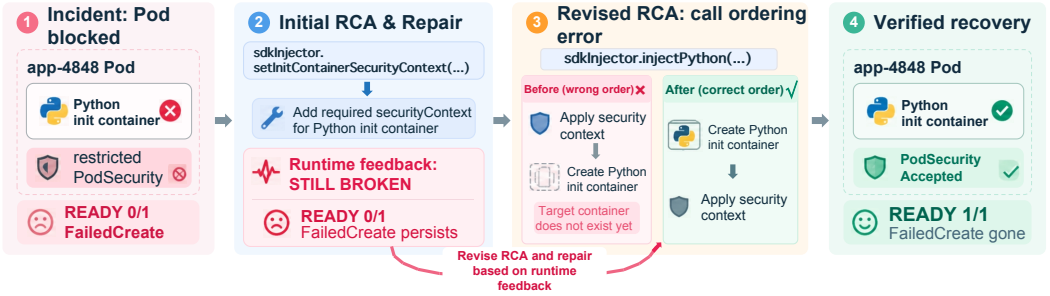


Fig. 8. Case A: An ineffective repair is corrected after the incident persists.

Figure 7(b) complements this trajectory view with a task-level comparison of diagnosis and system-side recovery. For Codex + GPT-5.6 Luna, 14.5% of tasks achieve recovery despite an incorrect joint RCA, whereas 20.0% have a correct joint RCA but do not recover. For Claude Code + Claude-Sonnet-5, the corresponding fractions are 21.8% and 7.3%. Reported RCA correctness and system-side recovery therefore only partially coincide: an incorrect reported fault location can still be followed by a successful repair, while a correct diagnosis does not guarantee successful remediation.

The case studies illustrate how diagnosis, repair, and verification interact during execution. In Case A (Figure 8), the initial repair remains ineffective after deployment, and the persistent failure leads the agent to revise its diagnosis and identify the ordering defect. In Case B (Figure 9), the agent eventually traces the authorization failure to the effective source manifest and repairs it, while its submitted RCA remains incorrect. Case C (Figure 10) follows the opposite pattern: repair removes immediate running-time symptom and restores workload, but fix-specific verification later reveals that the intended behavior was not preserved. Together, these cases show that intermediate diagnostic and repair outcomes do not uniquely determine final resolution: runtime feedback can correct an ineffective intervention, successful remediation can emerge despite an incorrect reported RCA, and apparent recovery can still fail functional verification.

5 Discussion

5.1 Implications

Maintain an explicit recovery contract. Our results suggest that incident-resolution agents should explicitly represent the behavior that a repair is expected to restore, rather than treating symptom disappearance as sufficient evidence of recovery. Such expectations can be derived from API definitions, configuration schemas, existing tests, and runtime observations, and should remain independent of candidate patches. This is particularly important for input-semantics defects, where removing an error may restore apparent availability while violating required behavior. An explicit recovery contract can therefore guide both repair generation and post-deployment verification.

Treat diagnosis and repair as a feedback-driven loop. The case studies show that root-cause hypotheses should remain revisable throughout remediation. An initially plausible diagnosis may lead to an ineffective repair and be corrected through runtime feedback, while an incorrect reported RCA may still be followed by a successful repair. Future agents should therefore use deployment outcomes as evidence for updating their hypotheses rather than treating RCA as a one-time stage. They should also trace whether a source modification is realized in the built artifact, deployed revision, and expected runtime effect, helping distinguish an incorrect hypothesis from an intervention that was never fully realized.

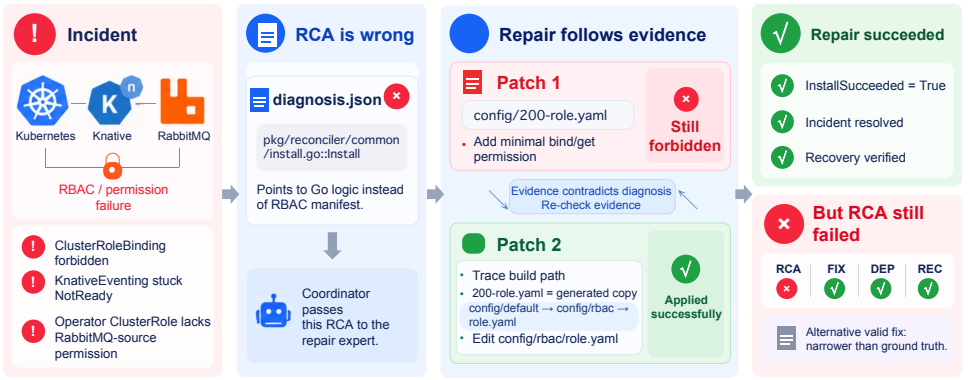


Fig. 9. Case B: Repair succeeds despite an incorrect RCA.

Target workflow mechanisms to observed failure modes. Different model–harness configurations exhibit different strengths, and substantially higher token consumption does not consistently yield better recovery. Together with frequent Action Realization Error and Insufficient Confirmation, this suggests that future workflows should target specific failure modes rather than uniformly adding more reasoning or tool use. For example, state tracking may reduce repeated investigation, while evidence-linked verification may reduce premature recovery declarations. Such mechanisms should be evaluated through matched-model and matched-budget comparisons.

5.2 Potential Impact

Supporting integrated development and operations research. SWEOps-Bench provides a common experimental setting for studying agents that operate across both source code and live cloud systems. By coupling runtime observations, source-level repair, deployment feedback, and verified recovery, it enables researchers to study how diagnosis and remediation interact during execution rather than evaluating them as isolated capabilities. This setting can support joint investigation by software engineering and systems communities into agents that must reason across code and operational state.

Turning historical defects into reusable research resources. Reproducing cloud incidents requires substantial effort to reconstruct software versions, dependencies, and triggering conditions. SWEOps-Bench packages confirmed historical defects into reusable executable environments that preserve their operational context and support controlled comparison of diagnosis and repair approaches. This reduces repeated reconstruction effort and provides a basis for extending evaluation to additional projects and newly confirmed defects.

5.3 Threats to Validity

Construct validity. Threats to construct validity concern whether our evaluation metrics accurately reflect incident resolution effectiveness and efficiency. In SWEOps-Bench, task recovery requires passing code compilation, container deployment, fault oracle clearance, and test suites. While these checks verify functional recovery under the triggering workload and guard against regressions, dynamic testing cannot mathematically prove behavioral correctness across every conceivable input condition. A candidate patch might clear the fault probe while leaving an unexercised corner case unresolved. Furthermore, the penalized latency metrics depend on predetermined stage weights (B, $3B$, and $6B$). These metrics integrate milestone completion rates with elapsed time, serving as standardized evaluation scores rather than literal production outage durations. Alternative penalty formulations could affect the relative efficiency rankings among agents.

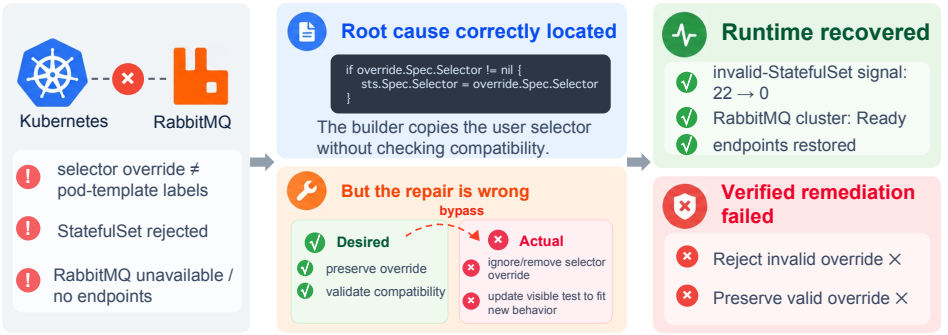


Fig. 10. Case C: Runtime symptoms disappear despite an incorrect repair.

Internal validity. Threats to internal validity relate to experimental confounding factors, stochastic generation, and potential LLM training data contamination. To reduce potential contamination from open-source repositories, SWEOps-Bench strictly withholds historical issue discussions and reference fixes from the execution environment. Additionally, 27 of our 55 tasks originate from contemporary 2026 pull requests, narrowing the potential pre-training exposure window for the evaluated models, although this does not establish the absence of prior exposure. The substantial computational overhead of repeatedly provisioning and executing live Kubernetes environments led us to evaluate one trial per task–configuration pair. Consequently, the reported recovery rates and lifecycle latencies characterize the observed trajectories rather than expectations over repeated stochastic executions. We therefore interpret differences among configurations descriptively rather than as statistically significant differences. Repeated trials could quantify run-to-run variance and assess whether the observed configuration-level differences remain stable across stochastic executions.

Conclusion validity. Threats to conclusion validity arise from the finite benchmark size and uneven task counts across incident categories. Although aggregate metrics cover all 55 tasks, several subgroup analyses in Figure 5 rely on relatively small numbers of cases. For example, Admission Policy contains three tasks, while Operator-reported and Storage blocked each contain five. Consequently, percentage differences within these categories may correspond to only one or two tasks. We therefore use the stratified results primarily to reveal heterogeneity across incident characteristics rather than to establish statistically stable differences between configurations. Future extensions with additional incidents per category and repeated executions could support confidence intervals and more robust statistical comparisons.

External validity. Threats to external validity concern the generalizability of our findings to other cloud environments and defect types. Our benchmark focuses on confirmed software defects within 16 representative Kubernetes operator projects. Consequently, our findings primarily characterize software-induced reconciliation failures in cloud management controllers, and may not directly generalize to infrastructure outages caused by hardware faults or to non-Kubernetes orchestration platforms. Furthermore, our selection methodology excludes non-deterministic races and silent state drifts lacking observable runtime anomalies.

6 Related Work

Section 2.3 compares SWEOps-Bench with existing operational and software repair benchmarks. This section contextualizes SWEOps-Bench within broader research on cloud incidents, automated failure diagnosis, and software repair.

Cloud incidents. Empirical studies of production cloud systems show that incidents often stem from complex software bugs, configuration drift, and unanticipated runtime interactions

rather than isolated hardware faults [5, 9, 45]. In cloud-native environments, the widespread adoption of softwarized management frameworks, such as Kubernetes operators, places complex reconciliation logic directly into operational loops [10, 11, 21, 30]. Defects in these controllers lead to runtime failures when reconciling desired and observed system state under dynamic workloads. Operational actions can restore service without correcting the defective implementation, making service recovery and implementation repair distinct concerns. SWEOps-Bench connects these concerns by reproducing confirmed historical operator defects and evaluating source or configuration repairs through their effects on the running system.

Automated failure diagnosis. Automated failure diagnosis uses runtime evidence to detect anomalies and identify their causes. Related methods and benchmarks examine metrics, logs, and distributed traces as sources of diagnostic information [28, 36, 41]. Telemetry-based techniques exploit service dependencies, execution traces, compact data representations, and learned failure patterns to localize faults [19, 20, 40]. LLM-based approaches combine diagnostic information collection with explanatory reasoning and tool-assisted investigation [7, 32]. Further work introduces procedural guidance, structured representations of Kubernetes state, and multi-agent fault diagnosis [27, 35, 44]. These approaches improve the ability to interpret operational evidence and produce diagnostic conclusions. Assessing a diagnosis, however, does not by itself establish that a subsequent repair restores the affected functionality. SWEOps-Bench evaluates this connection through patch generation, deployment, and recovery verification within the same incident-resolution process.

Software repair agents. Automated program repair investigates techniques to eliminate software defects with minimal human intervention. Search-based and constraint-based approaches use tests to guide candidate generation through genetic programming or semantic analysis [17, 25]. Learning-based repair incorporates project-specific knowledge and execution diagnostics, while conversational approaches use compilation and test feedback to revise unsuccessful candidates [34, 39]. Historical defect datasets and issue-resolution benchmarks provide reproducible subjects and executable tests for repair evaluation [15, 16]. Repository-level methods combine code search, contextual investigation, patch generation, and execution feedback through interactive or structured workflows [4, 33, 37, 43]. These methods already establish iterative repair and execution feedback as important capabilities. SWEOps-Bench evaluates these capabilities within an incident-resolution lifecycle that begins with observations from a running cluster. Candidate changes must be built and deployed, and their effects are assessed through runtime fault clearance, regression testing, and workload recovery.

7 Conclusion

This paper introduces SWEOps-Bench, an interactive benchmark for evaluating autonomous agents across the cloud incident resolution lifecycle. As autonomous agents advance, the boundary between cloud operations and software development diminishes, demanding integrated failure detection, localization, and source-level repair. Grounded in 55 reproducible historical defects from 16 Kubernetes operator projects, SWEOps-Bench combines interactive cluster environments with executable oracles covering runtime fault clearance and functional regression suites. Across six evaluated agent–model configurations, overall recovery ranges from 14.5% to 52.7%, showing that successful incident recognition and source-level diagnosis do not reliably translate into end-to-end resolution. Trajectory analysis further demonstrates that reported diagnosis and remediation are only partially coupled, and that runtime symptom clearance does not necessarily preserve the required functionality. By releasing the benchmark artifacts and evaluation infrastructure, SWEOps-Bench provides a foundation for developing more reliable agents in integrated development and operational settings.

8 Data Availability

The SWEOps-Bench benchmark and accompanying experimental artifacts are available in our anonymized repository at https://anonymous.4open.science/r/SWEOps-Bench-C221/.

References

[1] Atlassian. 2024. Incident Response: How We Respond to an Incident. Atlassian Incident Management Handbook. https://www.atlassian.com/incident-management/handbook/incident-response

[2] Algirdas Avizienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. 2004. Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE Trans. Dependable Secur. Comput.* 1, 1 (Jan. 2004), 11–33. doi:10.1109/TDSC.2004.2

[3] AXELOS. 2019. *ITIL Foundation: ITIL 4 Edition.* TSO (The Stationery Office).

[4] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2188–2200. doi:10.1109/ICSE55347.2025.00157

[5] Junjie Chen, Shu Zhang, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Yu Kang, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2020. How Incidental are the Incidents? Characterizing and Prioritizing Incidents for Large-Scale Online Service Systems. In *ASE 2020*. IEEE, 373–384. doi:10.1145/3324884.3416624

[6] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *MLSys 2025*. https://openreview.net/forum?id=3EXBLwGxtq

[7] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *EuroSys 2024*. ACM, 674–688. doi:10.1145/3627703.3629553

[8] Jackson Clark, Yiming Su, Saad Mohammad Rafid Pial, Lily Gniedziejko, and Tianyin Xu. 2026. SREGym: A Live Training Ground for AI SRE Agents with High-Fidelity Failure Drills. In *Proceedings of the ACM Conference on AI and Agentic Systems, CAIS 2026, San Jose, CA, USA, May 26-29, 2026*. ACM, 1253–1258. doi:10.1145/3786335.3813208

[9] Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. 2022. How to fight production incidents? an empirical study on a large-scale cloud service. In *Proceedings of the 13th Symposium on Cloud Computing* (San Francisco, California) *(SoCC '22)*. Association for Computing Machinery, New York, NY, USA, 126–141. doi:10.1145/3542929.3563482

[10] Jiawei Tyler Gu, Xudong Sun, Wentao Zhang, Yuxuan Jiang, Chen Wang, Mandana Vaziri, Owolabi Legunsen, and Tianyin Xu. 2023. Acto: Automatic End-to-End Testing for Operation Correctness of Cloud System Management. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 96–112. doi:10.1145/3600006.3613161

[11] Jiawei Tyler Gu, Zhen Tang, Yiming Su, Bogdan Alexandru Stoica, Xudong Sun, William X. Zheng, Yue Zhang, Akond Rahman, Chen Wang, and Tianyin Xu. 2026. Who Watches the Watchers? On the Reliability of Softwarizing Cloud Application Management. In *23rd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2026, Renton, WA, May 4-6, 2026*, Srikanth Kandula and Hakim Weatherspoon (Eds.). USENIX Association, 1171–1190. https://www.usenix.org/conference/nsdi26/presentation/gu

[12] Harbor Framework Team. 2026. *Harbor: A framework for evaluating and optimizing agents and models in container environments*. doi:10.5281/zenodo.20953922

[13] Frank Hutter, Holger H. Hoos, Kevin Leyton-Brown, and Thomas Stützle. 2009. ParamILS: An Automatic Algorithm Configuration Framework. *J. Artif. Intell. Res.* 36 (2009), 267–306. doi:10.1613/JAIR.2861

[14] Saurabh Jha, Rohan R. Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya, Mudit Verma, Harshit Kumar, Hirokuni Kitahara, Noah Zheutlin, Saki Takano, Divya Pathak, Felix George, Xinbo Wu, Bekir O. Turkkan, Gerard Vanloo, Michael Nidd, Ting Dai, Oishik Chatterjee, Pranjal Gupta, Suranjana Samanta, Pooja Aggarwal, Rong Lee, Jae-wook Ahn, Debanjana Kar, Amit M. Paradkar, Yu Deng, Pratibha Moogi, Prateeti Mohapatra, Naoki Abe, Chandrasekhar Narayanaswami, Tianyin Xu, Lav R. Varshney, Ruchi Mahindru, Anca Sailer, Laura Shwartz, Daby Sow, Nicholas C. Fuller, and Ruchir Puri. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. In *ICML 2025*. OpenReview.net. https://openreview.net/forum?id=jP59rz1bZk

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net. https://openreview.net/forum?id=VTF8yNQM66

[16] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA,*

USA - July 21 - 26, 2014, Corina S. Pasareanu and Darko Marinov (Eds.). ACM, 437–440. doi:10.1145/2610384.2628055

[17] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Trans. Softw. Eng.* 38, 1 (Jan. 2012), 54–72. doi:10.1109/TSE.2011.104

[18] Xiaoyun Li, Guangba Yu, Pengfei Chen, Hongyang Chen, and Zhekang Chen. 2022. Going through the Life Cycle of Faults in Clouds: Guidelines on Fault Handling. In *ISSRE 2022*. IEEE, 121–132. doi:10.1109/ISSRE55969.2022.00022

[19] Yufeng Li, Guangba Yu, Pengfei Chen, Chuanfu Zhang, and Zibin Zheng. 2022. MicroSketch: Lightweight and Adaptive Sketch Based Performance Issue Detection Localization in Microservice Systems. In *ICSOC 2022*. Springer-Verlag, Berlin, Heidelberg, 219–236. doi:10.1007/978-3-031-20984-0_15

[20] Zeyan Li, Nengwen Zhao, Mingjie Li, Xianglin Lu, Lixin Wang, Dongdong Chang, Xiaohui Nie, Li Cao, Wenchi Zhang, Kaixin Sui, Yanhua Wang, Xu Du, Guoqiang Duan, and Dan Pei. 2022. Actionable and interpretable fault localization for recurring failures in online service systems. In *ESEC/FSE 2022*. ACM, 996–1008. doi:10.1145/3540250.3549092

[21] Bingzhe Liu, Gangmuk Lim, Ryan Beckett, and P. Brighten Godfrey. 2024. Kivi: verification for cluster management. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference* (Santa Clara, CA, USA) *(USENIX ATC'24)*. USENIX Association, USA, Article 32, 19 pages.

[22] Yuhe Liu, Changhua Pei, Longlong Xu, Bohan Chen, Mingze Sun, Zhirui Zhang, Yongqian Sun, Shenglin Zhang, Kun Wang, Haiming Zhang, Jianhui Li, Gaogang Xie, Xidao Wen, Xiaohui Nie, Minghua Ma, and Dan Pei. 2025. OpsEval: A Comprehensive Benchmark Suite for Evaluating Large Language Models' Capability in IT Operations Domain. In *FSE Companion 2025*. ACM, 503–513. doi:10.1145/3696630.3728572

[23] LogicMonitor. 2024. What's the Difference Between MTTR, MTBF, MTTD, and MTTF. LogicMonitor Blog. https://www.logicmonitor.com/blog/whats-the-difference-between-mttr-mttd-mttf-and-mtbf

[24] Mike A. Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, Estefany Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis-Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*, Carl Vondrick, Bharath Hariharan, Colin Raffel, Lerrel Pinto, Diyi Yang, and Aleksandra Faust (Eds.). proceedings.iclr.cc. https://proceedings.iclr.cc/paper_files/paper/2026/hash/444a3737adaee10d86ad2ef5f74468e6-Abstract-Conference.html

[25] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: program repair via semantic analysis. In *Proceedings of the 2013 International Conference on Software Engineering* (San Francisco, CA, USA) *(ICSE '13)*. IEEE Press, 772–781.

[26] OT-CONTAINER-KIT. 2026. fix(utils): pass client certificate to exec-based redis-cli. GitHub pull request #1864. https://github.com/OT-CONTAINER-KIT/redis-operator/pull/1864

[27] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, and Dan Pei. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *WWW 2025*. Association for Computing Machinery, 422–431. doi:10.1145/3701716.3715225

[28] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *WWW 2025*. ACM, 777–780. doi:10.1145/3701716.3715290

[29] Redis-Operator. 2025. RedisCluster Pods Aren't Created with No Persistence and Using initContainer. GitHub issue #1540. https://github.com/OT-CONTAINER-KIT/redis-operator/issues/1540

[30] Xudong Sun, Wenqing Luo, Jiawei Tyler Gu, Aishwarya Ganesan, Ramnatthan Alagappan, Michael Gasch, Lalith Suresh, and Tianyin Xu. 2022. Automatic Reliability Testing For Cluster Management Controllers. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 143–159. https://www.usenix.org/conference/osdi22/presentation/sun

[31] Yilun Wang, Guangba Yu, Haiyu Huang, Yujie Huang, Zirui Wang, Pengfei Chen, and Michael R. Lyu. 2026. CloudOpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. arXiv:2603.00468 [cs.SE] https://arxiv.org/abs/2603.00468

[32] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large

Language Models. In *CIKM 2024*. ACM, 4966–4974. doi:10.1145/3627673.3680016

[33] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* 2, FSE (2025), 801–824. doi:10.1145/3715754

[34] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. doi:10.1145/3650212.3680323

[35] Yong Xiang, Charley Peter Chen, Liyi Zeng, Wei Yin, Xin Liu, Hu Li, and Wei Xu. 2025. Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM. *CoRR* abs/2506.02490 (2025). arXiv:2506.02490 doi:10.48550/ARXIV.2506.02490

[36] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *ICLR 2025*. OpenReview.net. https://openreview.net/forum?id=M4qNIzQYpd

[37] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601

[38] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. 2025. SWE-bench Multimodal: Do AI Systems Generalize to Visual Software Domains?. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=riTiq3i21b

[39] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics. In *37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022*. ACM, 92:1–92:13. doi:10.1145/3551349.3556926

[40] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *WWW 2021*. ACM, 3087–3098. doi:10.1145/3442381.3449905

[41] Guangba Yu, Pengfei Chen, Yufeng Li, Hongyang Chen, Xiaoyun Li, and Zibin Zheng. 2023. Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. In *ESEC/FSE 2023*. ACM, 553–565. doi:10.1145/3611643.3616249

[42] Guangba Yu, Genting Mai, Rui Wang, Ruipeng Li, Pengfei Chen, Long Pan, and Ruijie Xu. 2026. AlertGuardian: Intelligent Alert Life-Cycle Management for Large-scale Cloud Systems. arXiv:2601.14912 [cs.DC] https://arxiv.org/abs/2601.14912

[43] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1592–1604. doi:10.1145/3650212.3680384

[44] Yuedong Zhong, Guangba Yu, Yujie Huang, QunChao Fu, Rui Ren, Cong Feng, Yongqiang Yang, and Michael Lyu. 2026. LLMGuard: Multi-Agent Fault Diagnosis for Reliable Language-Model-as-a-Service. In *2026 56th Annual IEEE International Conference on Dependable Systems and Networks (DSN)*. 15–23. doi:10.1109/DSN69566.2026.00018

[45] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Software Eng.* 47, 2 (2021), 243–260. doi:10.1109/TSE.2018.2887384